

Web Application Security Assessment

Northstar Ticketing Application

Assessment type: Authorized web application security assessment

Prepared by: Arky Craft
Date: September 2026

This report documents the methodology, findings, and recommendations from a targeted security assessment of the Northstar web application in a controlled lab environment.

Contents

Contents	2
1. Executive Summary.....	3
2. Scope and Rules	4
3. Environment and Methodology	5
4. Findings Summary.....	7
5. Detailed Findings	8
F-01: Insecure Direct Object Reference (IDOR) in Ticket Detail Endpoint.....	8
F-02: Incomplete Session-Cookie Security Attributes and HTTP Transport	10
T-01: SQL Injection Testing of Login Endpoint — No Exploitation Observed	11
T-02: Vertical Privilege Escalation — No Exploitation Observed	12
T-03: Reflected XSS Testing — No Exploitation Observed	13
T-04: CSRF Testing Limitations.....	14
T-05: Password-Storage Review Limitation	14
6. Remediation Roadmap	15
7. Conclusion.....	15
Appendix A: Evidence and Screenshots	16
Appendix B: Test Limitations.....	29
Appendix C: References.....	31

1. Executive Summary

The assessor carried out a web application security assessment on the Northstar Support application, a Flask-based training environment hosted at `http://192.168.XXX.XXX:5000`. Testing focused on authentication, authorization, session management, input handling, error handling, security configuration, and monitoring using Burp Suite and manual verification techniques aligned with the OWASP Web Security Testing Guide (WSTG).

The assessment identified one confirmed **high-severity** broken-access-control vulnerability. A standard authenticated user could access another user's support ticket by changing the numeric ticket identifier in the `/tickets/<id>` route. For example, an authenticated session belonging to Alice accessed Dana's ticket through `GET /tickets/2`, even though Ticket 2 was assigned to Dana. This represents a horizontal privilege-escalation / insecure direct object reference (IDOR) issue.

The assessment also identified incomplete session-cookie hardening. The observed session cookie included the `'HttpOnly'` attribute but did not visibly include `'Secure'` or `'SameSite'`; the lab application was served over plain HTTP. SQL injection tests against the login endpoint, reflected-XSS testing of available login inputs, and vertical privilege-escalation testing against the administrator endpoint did not result in exploitation. The available standard-user functionality did not permit a full assessment of password-storage strength, stored XSS, or meaningful CSRF exploitability.

Overall risk rating: High. The confirmed ticket IDOR permits unauthorized disclosure of other users' support-ticket content.

Key Recommendations

- Enforce server-side object-level authorization for every ticket request, edit, attachment, comment, and API operation.
- Enable HTTPS and configure `Secure`, `HttpOnly`, and `SameSite` session-cookie attributes.
- Use centralized role and ownership checks rather than relying on UI visibility or route naming.
- Add authorization regression tests with multiple user roles and distinct standard-user accounts.
- Add application-level logging for authentication failures and authorization denials without storing passwords or session tokens.

2. Scope and Rules

2.1 In-Scope Assets

<u>Asset</u>	<u>Scope</u>
Northstar Support web application	http://192.168.XXX.XXX:5000
Authentication workflow	POST /login, logout/session behavior
User profile functionality	/user/<id>
Ticket functionality	/tickets/<id>
Administrator endpoint	/admin/dana
Backend database	Local SQLite database used by the authorized lab
Application logs	Local lab/server logs, where available

2.2 Out-of-Scope Activities

- Network infrastructure beyond the target web server
- Denial-of-service testing
- Social engineering
- Physical security testing
- Testing against external systems or real user data
- Production environments
- Unauthorized account access

2.3 Rules of Engagement

- The assessor performed testing only in an isolated, assessor-controlled training lab.
- The assessor only utilized authorized test accounts and fictional lab data.
- They limited testing to manual web-application security verification and controlled Burp Suite requests.
- The assessor did not carry out any destructive data modification or denial-of-service activity.
- They sanitized the evidence before including it in the final portfolio report.

3. Environment and Methodology

3.1 Lab Environment

<u>Component</u>	<u>Details</u>
Tester workstation	Kali Linux
Target system	Ubuntu VM hosting Northstar Support
Application framework	Python / Flask
Web service	HTTP on port 5000
Database	SQLite (instance/supporthub.sqlite)
Interception proxy	Burp Suite Community Edition
Browser	Firefox configured through Burp proxy
Burp listener	127.0.0.1:8080
Target scope	http://192.168.XXX.XXX:5000

3.2 Tools Used

<u>Tool</u>	<u>Purpose</u>
Burp Suite Community Edition	Intercepted, inspected, and modified HTTP requests and responses. Used Proxy, HTTP History, Site Map, Repeater, Intruder, and session-management features to support authentication, authorization, session, and input-validation testing.
Mozilla Firefox	Accessed the Northstar application and generated browser traffic through the Burp proxy.
Visual Studio Code	Used as the integrated development environment (IDE) for reviewing and maintaining the Flask application source code, Python scripts, HTML templates, configuration files, and SQLite database-related project files.
Visual Studio Code CLI	Used to run Linux shell commands, activate the Python virtual environment, start and stop the Flask application, execute supporting Python scripts, inspect application output, and run SQLite commands.
SQLite CLI (sqlite3)	Used for authorized local database inspection and verification of user and ticket ownership records supporting the IDOR test.

Python / Flask	Application technology assessed in the lab environment; Python scripts and Flask application components were reviewed and executed as part of the assessor-controlled lab workflow.
Linux command-line utilities	Used for authorized file navigation, process review, and log inspection in the local lab environment.

3.3 Methodology

Testing followed an OWASP-aligned workflow:

1. Reconnaissance and mapping

- Configured Burp Proxy and target scope.
- Browsed the application and reviewed the Burp Site Map and HTTP history.
- Identified authentication, ticket, profile, dashboard, logout, and admin routes.

2. Authentication testing

- Verified valid-login behavior.
- Tested invalid usernames and invalid passwords.
- Performed limited password-list testing with Burp Intruder.
- Performed controlled SQL-injection probes in the login parameters.

3. Authorization testing

- Tested horizontal authorization by modifying ticket identifiers.
- Tested profile-object access where available.
- Tested vertical authorization by requesting the admin endpoint as a standard user.

4. Session-management testing

- Inspected Set-Cookie attributes.
- Tested arbitrary invalid-cookie rejection.
- Reviewed logout behavior.
- Configured Burp session handling to maintain the authorized test session.

5. Input validation and error handling

- Tested login parameters for SQL injection and basic reflected-XSS markers.
- Reviewed invalid-route responses for verbose errors and stack traces.
- Inspected headers for security-hardening controls.

6. Logging and monitoring

- Generated controlled failed-login and unauthorized-admin-access events.
- Reviewed accessible server/application logs, where available.

4. Findings Summary

<u>ID</u>	<u>Finding</u>	<u>Severity</u>	<u>OWASP Category</u>	<u>Status</u>
F-01	Insecure Direct Object Reference (IDOR) in Ticket Detail Endpoint	High	A01:2025—Broken Access Control	Confirmed
F-02	Incomplete Session-Cookie Security Attributes and HTTP Transport	Medium	A07:2025—Authentication Failures	Confirmed
T-01	SQL Injection Testing of Login Endpoint	Informational	A05:2025—Injection	No exploitation observed
T-02	Vertical Privilege Escalation at Admin Endpoint	Informational	A01:2025—Broken Access Control	No exploitation observed
T-03	Reflected XSS Testing of Login Endpoint	Informational	A05:2025—Injection	No exploitation observed
T-04	CSRF Testing Limitations	Informational	Session Management	Not fully testable
T-05	Password Storage Review	Informational	A07:2025—Authentication Failures	Not assessed / insufficient evidence

5. Detailed Findings

F-01: Insecure Direct Object Reference (IDOR) in Ticket Detail Endpoint

Severity: High

Affected component: GET /tickets/<id>

OWASP Top 10 mapping: A01:2025—Broken Access Control

WSTG mapping: Authorization / IDOR testing

Description

The assessor identified a horizontal privilege-escalation vulnerability in the Northstar ticket-detail endpoint. While authenticated as Alice, a standard user, the assessor first accessed Alice's own ticket at:

```
GET /tickets/1
```

The assessor retained the same session, user identity, and role and changed only the numeric object identifier to:

```
GET /tickets/2
```

The application returned Dana's ticket, including ticket title, description, priority, status, owner identifier, assigned identifier, and timestamps. Database verification confirmed that Ticket 1 belonged to Alice (user ID 1) and Ticket 2 belonged to Dana (user ID 2). This demonstrates that the application did not validate ticket ownership before returning the resource.

Proof of Concept

1. The assessor authenticated as Alice using a valid standard-user account.
2. The assessor accessed `http://192.168.XXX.XXX:5000/tickets/1` and confirmed that Ticket 1 belonged to Alice.
3. The authenticated session remained the same.
4. The assessor changed the ticket identifier to request `http://192.168.XXX.XXX:5000/tickets/2`.
5. The application returned Dana's ticket with HTTP 200 OK rather than returning 403 Forbidden or 404 Not Found.
6. The assessor reviewed the local authorized SQLite database to confirm that Ticket 1 belonged to Alice and Ticket 2 belonged to Dana.

Impact

Any authenticated user may be able to enumerate and access ticket records belonging to other users. In a real support environment, this could expose confidential customer information, support requests, internal troubleshooting information, account-related details, and potentially security-sensitive content.

Remediation

Implement a server-side authorization check before returning a ticket:

```
ticket = get_ticket_by_id(ticket_id)

if ticket is None:
    abort(404)

if ticket["owner_user_id"] != session.get("user_id"):
    abort(403)
```

- Apply authorization checks to ticket views, edits, comments, file attachments, downloads, APIs, and deletion actions.
- Enforce authorization on the server for every request.
- Do not rely on hidden links, unguessable identifiers, or client-side checks.
- Create automated regression tests using at least two normal users and one administrator.
- Log denied cross-user access attempts without exposing sensitive ticket data.

Evidence (see Appendix A.4)

- Figure 11a: Alice accessed her own ticket at `/tickets/1.`
- Figure 11b. Alice's ticket list (baseline view).
- Figure 12. Alice accessed Dana's ticket after the ticket identifier was changed to `/tickets/2.`
- Figure 13. Burp Repeater evidence of cross-user ticket disclosure.
- Figure 14. SQLite user and ticket ownership mapping.

F-02: Incomplete Session-Cookie Security Attributes and HTTP Transport

Severity: Medium

Affected component: Session cookie issued after POST /login

OWASP Top 10 mapping: A07:2025—Authentication Failures

WSTG mapping: Session-management testing

Description

The captured response to a successful login set a session cookie containing the `HttpOnly` attribute and `Path=/. The observed Set-Cookie header did not visibly include Secure or SameSite. The assessor accessed the lab application through unencrypted HTTP.`

`HttpOnly` is a positive control because it limits direct access to the cookie from client-side JavaScript. However, the absence of `Secure` allows the browser to transmit the cookie over HTTP, and the lack of `SameSite` removes an important browser-level defense-in-depth control against cross-site request risks.

The login response redirected the user to /dashboard using 302 Found and issued a session cookie with `HttpOnly; Path=/. Secure and SameSite were not visible in the captured response header.`

Impact

Deploying the same configuration on an untrusted or shared network would allow an attacker observing HTTP traffic to potentially capture an authenticated session cookie. Missing `SameSite` also reduces browser-based defense in depth against CSRF if state-changing features are introduced.

Remediation

Enable HTTPS in deployed environments and configure secure session settings:

```
SESSION_COOKIE_HTTPONLY = True
SESSION_COOKIE_SECURE = True
SESSION_COOKIE_SAMESITE = "Lax"
```

Use `SameSite="Strict"` where application workflows permit it. Also:

- Configure an idle timeout and an absolute session lifetime.
- Regenerate session identifiers after authentication and privilege changes.
- Do not place session tokens in URLs.
- Use server-side session invalidation or an appropriate token revocation strategy on logout.

Evidence (see Appendix A.3)

- Figure 9. Set-Cookie header showing session cookie name and attributes.
- Figures 10a–10c. Session binding, tamper resistance, and logout behavior.

T-01: SQL Injection Testing of Login Endpoint — No Exploitation Observed

Severity: Informational

Affected component: POST /login

OWASP Top 10 mapping: A05:2025—Injection

WSTG mapping: SQL injection testing

Description

The assessor tested the login email and password parameters through Burp Repeater using controlled SQL syntax and Boolean-style probes, including:

```
'  
' OR '1'='1  
' AND '1'='2
```

The application consistently returned the generic response:

```
Invalid email or password.
```

No successful authentication occurred, no redirect to /dashboard was observed for invalid test values, no authenticated session was established, and no database error messages were disclosed.

Conclusion

The assessment did not demonstrate any exploitable SQL injection or authentication bypass on the tested login endpoint.

Recommendation

- Continue using parameterized queries for all database interactions.
- Avoid dynamic SQL construction through string concatenation.
- Keep generic authentication errors.
- Add monitoring, rate limiting, and account-lockout controls appropriate to the deployment environment.

T-02: Vertical Privilege Escalation — No Exploitation Observed

Severity: Informational

Affected component: GET /admin/dana

OWASP Top 10 mapping: A01:2025—Broken Access Control

WSTG mapping: Privilege-escalation testing

Description

The assessor requested the administrator endpoint while using Alice’s authenticated standard-user session:

```
GET /admin/dana HTTP/1.1
Host: 192.168.XXX.XXX:5000
Cookie: session=abcdef123456
```

The server returned:

```
HTTP/1.1 403 Forbidden
```

No administrative content or privileged functions were disclosed.

Conclusion

When Alice’s authenticated standard-user session requested GET /admin/dana, the server returned HTTP 403 Forbidden and did not disclose administrative content or privileged functionality.

Recommendation

Continue enforcing centralized server-side role checks on all administrator routes, backend APIs, and privileged actions. Add automated tests confirming that standard users receive 403 Forbidden when requesting administrator-only functionality.

T-03: Reflected XSS Testing — No Exploitation Observed

Severity: Informational

Affected component: POST /login

OWASP Top 10 mapping: A05:2025—Injection

WSTG mapping: Reflected-XSS testing

Description

The assessor used an HTML marker to test the login email parameter in Burp Repeater, such as:

```
<xss-check>
```

The assessor submitted the marker in URL-encoded form:

```
email=%3Cxss-check%3E
```

The application returned a generic failed-authentication message and did not reflect the submitted marker in the response.

Conclusion

The tested login endpoint did not demonstrate any reflected XSS.

Limitation

The assessment did not include stored-XSS testing because the available standard-user interface did not expose a persistent user-content workflow, such as ticket creation, ticket editing, comments, or profile editing.

Recommendation

Maintain context-appropriate output encoding for all user-controlled values. Ensure Flask/Jinja automatic escaping remains enabled and only mark user input as safe after rigorous sanitization for the specific output context.

T-04: CSRF Testing Limitations

Severity: Informational

Affected component: Standard-user functionality

OWASP mapping: Session-management / CSRF testing

Description

The assessor reviewed CSRF protections during the session-management assessment. A meaningful CSRF proof of concept requires an authenticated, state-changing endpoint, such as ticket creation, profile modification, password change, or record deletion.

The available standard-user Northstar interface exposed login and read-only ticket/profile functions but did not expose a suitable authenticated state-changing workflow.

Conclusion

The assessment could not establish CSRF exploitability because the assessed functionality lacked a suitable authenticated, state-changing standard-user endpoint.

Recommendation

When the application adds state-changing functionality, protect all such requests by using server-side CSRF tokens and validation. Use SameSite cookie attributes as an additional defense layer rather than relying on them as the sole control.

T-05: Password-Storage Review Limitation

Severity: Informational

Affected component: User credential storage

OWASP mapping: A07:2025—Authentication Failures

Description

The assessment did not obtain sufficient evidence to determine the password-hashing algorithm, salt behavior, cost parameters, or password-verification implementation.

The presence of a user database alone does not establish whether the application stores passwords in plaintext, uses weak hashing such as MD5 or SHA-1, or uses an adaptive password-hashing algorithm such as bcrypt, scrypt, PBKDF2, or Argon2id.

Conclusion

The assessment did not cover password-storage strength. This report does not include any findings related to weak password hashing.

Recommendation

Perform an authorized source-code or database-format review. Developers should never store passwords in plaintext or use reversible encryption. Use modern, slow, adaptive password-hashing algorithms—Argon2id is recommended, with scrypt, bcrypt, or PBKDF2 as suitable alternatives depending on deployment requirements.

6. Remediation Roadmap

<u>Priority</u>	<u>Timeframe</u>	<u>Action</u>
Immediate	0–2 weeks	Fix the ticket IDOR through server-side object ownership and authorization checks
Immediate	0–2 weeks	Add regression tests for cross-user ticket viewing, editing, comments, and attachments
Short term	2–4 weeks	Enable HTTPS and set Secure, HttpOnly, and SameSite cookie attributes
Short term	2–4 weeks	Review all routes for centralized role-based and ownership-based authorization
Medium term	1–3 months	Add CSRF protection to all state-changing workflows
Medium term	1–3 months	Improve application-level monitoring for failed logins and authorization denials
Medium term	1–3 months	Perform a source/database review of password storage and password-reset functionality
Ongoing	Continuous	Repeat web-application security testing after remediation and before deployment

7. Conclusion

The Northstar Support application demonstrated a significant authorization weakness in ticket access controls. The confirmed ticket IDOR permits one authenticated user to access another user’s ticket record by modifying an identifier in the request URL. The development team should remediate this issue before the application is considered suitable for a production-like environment.

The admin endpoint correctly denied a standard-user session with 403 Forbidden, and the tested login endpoint did not demonstrate SQL injection or reflected XSS. However, consistent authorization enforcement is required across all object-specific routes, as demonstrated by the difference between the protected admin endpoint and the vulnerable ticket endpoint.

The application should prioritize object-level authorization, HTTPS deployment, session-cookie hardening, security-event logging, and follow-up testing after remediation.

Appendix A: Evidence and Screenshots

This appendix lists the key evidence collected during the assessment and maps each finding or test case to its supporting screenshots. All images were captured from Burp Suite or the browser during testing in the lab environment.

A.1 Reconnaissance and Scope

Proxy configuration and target scope

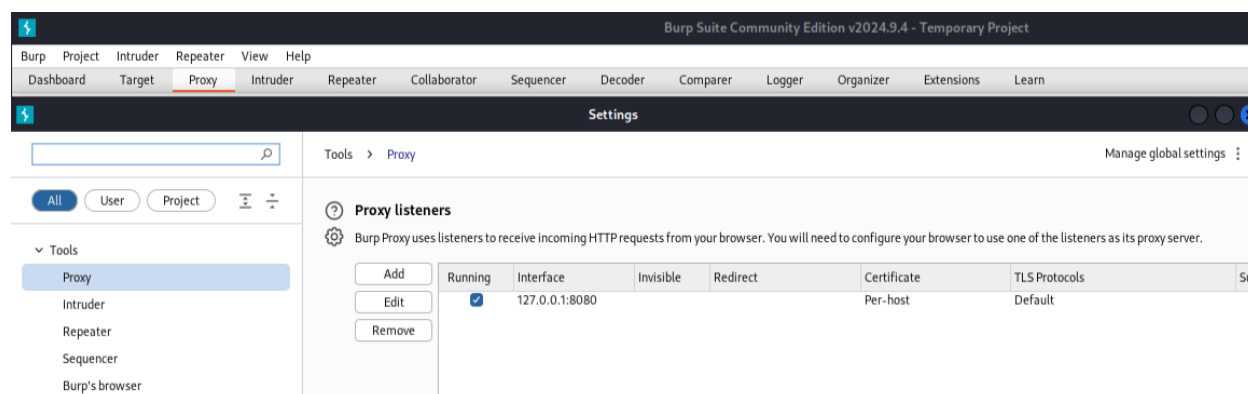


Figure 1. Burp proxy listener configuration

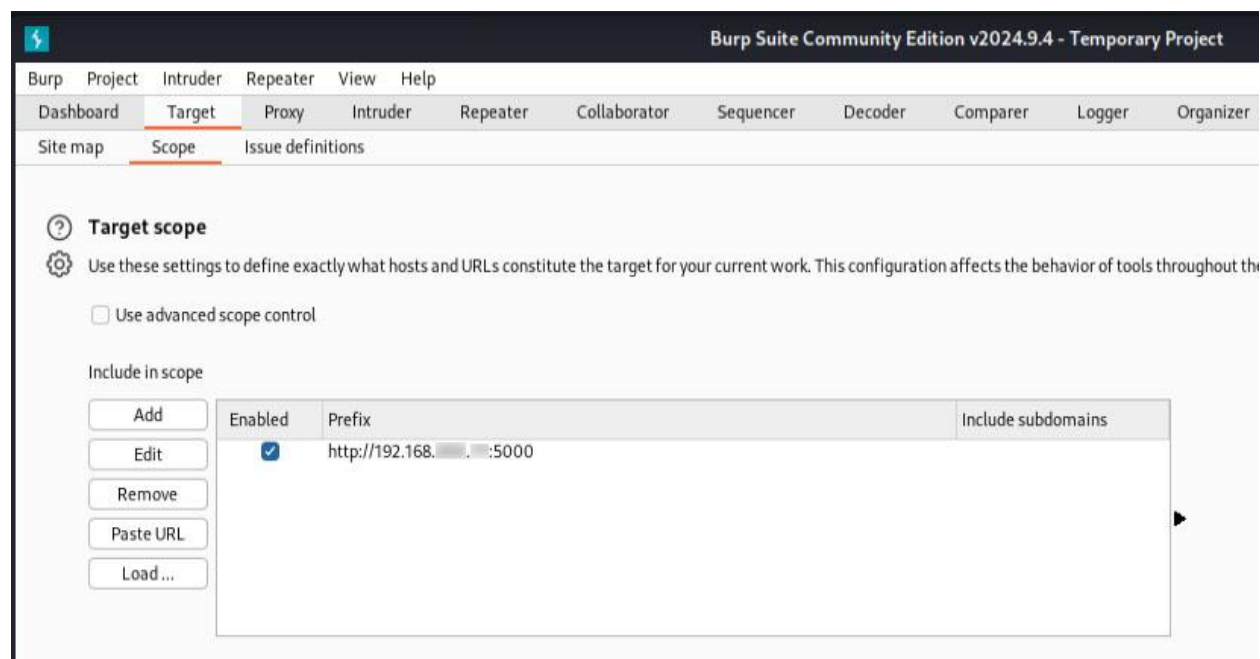


Figure 2. Defined target scope in Burp to constrain testing to authorized hosts

Application mapping

The screenshot displays the Burp Suite Community Edition v2024.9.4 interface. The top menu bar includes options like Project, Intruder, Repeater, View, Help, Dashboard, Target, Proxy, Intruder, Repeater, Collaborator, Sequencer, Decoder, Comparer, Logger, Organizer, Extensions, and Learn. The 'Site map' tab is active, showing a tree view of the application structure on the left. The tree view includes folders for 'http://192.168.1.100:5000', 'login', 'logout', 'static', 'css', 'js', 'app.js', 'user', and '1'. The 'login' folder is expanded, showing a list of discovered paths, resources, and application structure. The table below lists the discovered paths, resources, and application structure.

Host	Method	URL	Params	Status code	Length	MIME type	Title	Notes	Time requested
http://192.168.1.100:5000	POST	/login		200	1794	HTML	Sign in — Northstar Support		17:44:06 30 Aug 2026
http://192.168.1.100:5000	POST	/login		302	513	HTML	Redirecting...		17:44:34 30 Aug 2026
http://192.168.1.100:5000	GET	/logout		302	483	HTML	Redirecting...		17:44:48 30 Aug 2026
http://192.168.1.100:5000	GET	/		200	1824	HTML	Northstar Support		17:44:56 30 Aug 2026
http://192.168.1.100:5000	GET	/about		200	1276	HTML	About — Northstar Support		17:45:04 30 Aug 2026
http://192.168.1.100:5000	GET	/user/1		200	1304	HTML	User Profile — Northstar Support		17:45:14 30 Aug 2026
http://192.168.1.100:5000	GET	/user/2		200	1302	HTML	User Profile — Northstar Support		17:45:20 30 Aug 2026
http://192.168.1.100:5000	GET	/login		200	1733	HTML	Sign in — Northstar Support		17:46:02 30 Aug 2026
http://192.168.1.100:5000	POST	/login		200	1794	HTML	Sign in — Northstar Support		17:46:29 30 Aug 2026
http://192.168.1.100:5000	POST	/login		302	513	HTML	Redirecting...		17:47:03 30 Aug 2026
http://192.168.1.100:5000	GET	/dashboard		200	1801	HTML	Dashboard — Northstar Support		17:47:05 30 Aug 2026
http://192.168.1.100:5000	GET	/static/css/styles.css		304	314				17:47:05 30 Aug 2026
http://192.168.1.100:5000	GET	/static/js/app.js		304	309				17:47:05 30 Aug 2026

The bottom section of the screenshot shows the 'Request' and 'Response' tabs. The 'Request' tab displays a POST request to /login with various headers and a body containing email and password fields. The 'Response' tab displays the corresponding HTML response, which includes a sign-in form and an error message: 'Invalid email or password.'

Figure 3. Burp site map showing discovered paths, resources, and application structure

A.2 Authentication Testing

Valid login flow

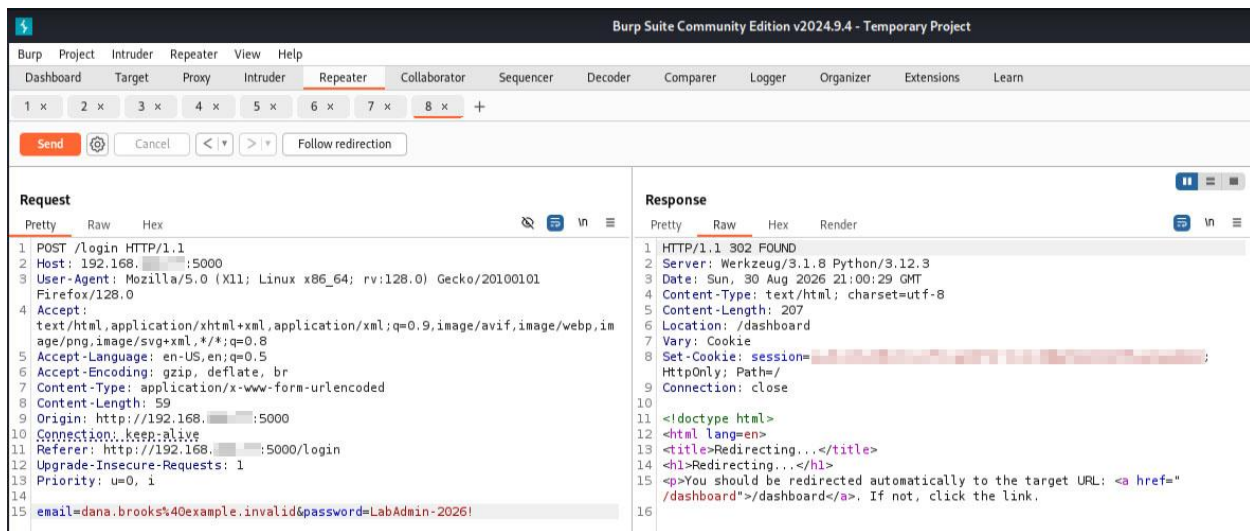


Figure 4. Successful login request and response using valid credentials

Invalid credentials handling

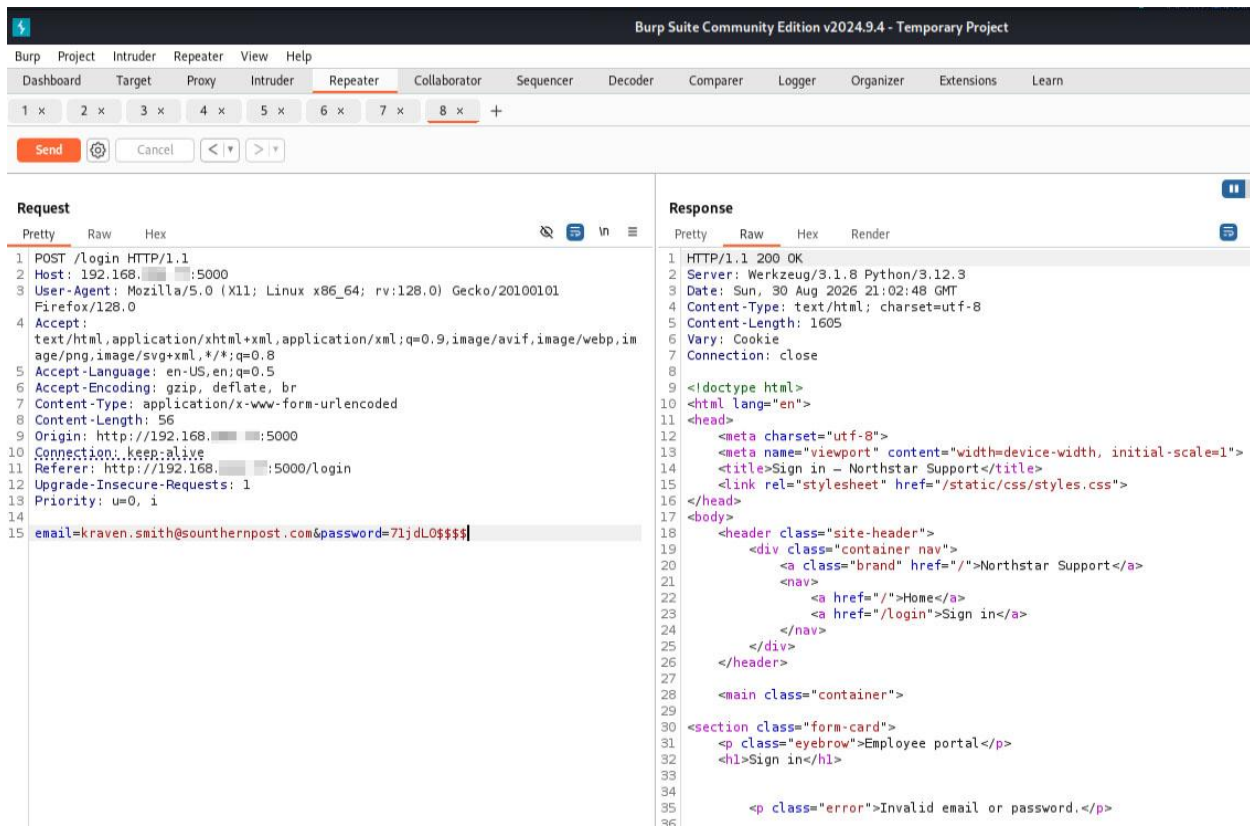


Figure 5. Login attempt with a non-existent email address


 Burp Suite Community Edition v2024.9.4 - Temporary Project

Burp Project Intruder Repeater View Help
 Dashboard Target Proxy Intruder **Repeater** Collaborator Sequencer Decoder Comparer Logger Organizer Extensions Learn

1 x 2 x 3 x 4 x 5 x 6 x 7 x **8 x** +

Send  Cancel < >

Request

Pretty Raw Hex   

```

1 POST /login HTTP/1.1
2 Host: 192.168.1.1:5000
3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101
  Firefox/128.0
4 Accept:
  text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,im
  age/png,image/svg+xml,*/*;q=0.8
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Content-Type: application/x-www-form-urlencoded
8 Content-Length: 54
9 Origin: http://192.168.1.1:5000
10 Connection: keep-alive
11 Referer: http://192.168.1.1:5000/login
12 Upgrade-Insecure-Requests: 1
13 Priority: u=0, i
14
15 email=dana.brooks%40example.invalid&password=987654321
            
```

Response

Pretty Raw Hex Render   

```

1 HTTP/1.1 200 OK
2 Server: Werkzeug/3.1.8 Python/3.12.3
3 Date: Sun, 30 Aug 2026 20:58:07 GMT
4 Content-Type: text/html; charset=utf-8
5 Content-Length: 1605
6 Vary: Cookie
7 Connection: close
8
9 <!doctype html>
10 <html lang="en">
11 <head>
12   <meta charset="utf-8">
13   <meta name="viewport" content="width=device-width, initial-scale=1">
14   <title>Sign in - Northstar Support</title>
15   <link rel="stylesheet" href="/static/css/styles.css">
16 </head>
17 <body>
18   <header class="site-header">
19     <div class="container nav">
20       <a class="brand" href="/">Northstar Support</a>
21       <nav>
22         <a href="/">Home</a>
23         <a href="/login">Sign in</a>
24       </nav>
25     </div>
26   </header>
27
28   <main class="container">
29
30     <section class="form-card">
31       <p class="eyebrow">Employee portal</p>
32       <h1>Sign in</h1>
33
34
35       <p class="error">Invalid email or password.</p>
36
37
38       <form method="post" action="/login">
39         <label for="email">Email address</label>
40         <input
41           id="email"
42           name="email"
43           type="email"
44           autocomplete="username"
            
```

Figure 6. Login attempt with an incorrect password

Password brute-force feasibility (limited)

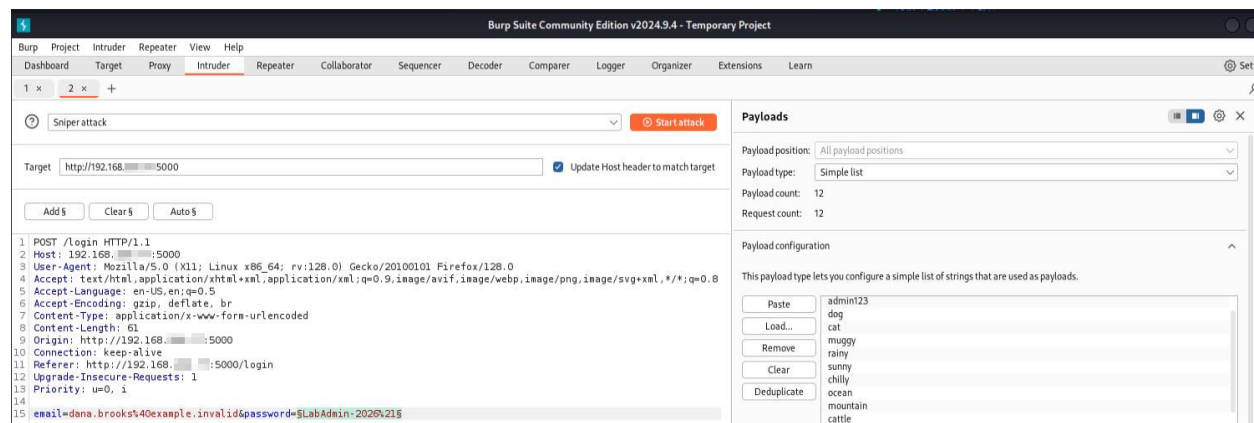


Figure 7. Intruder payload configuration targeting the password field

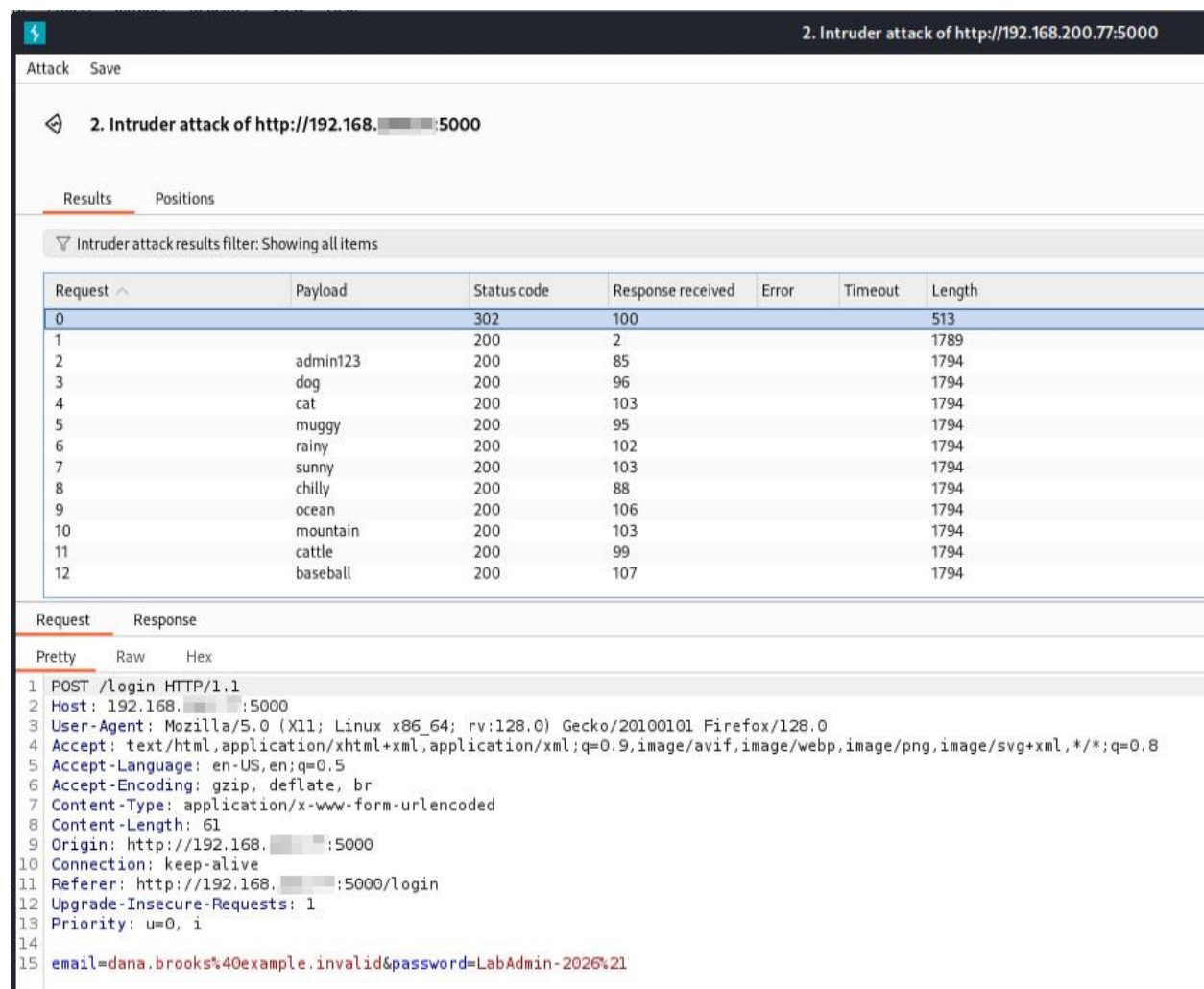


Figure 8. Intruder results table for password brute-force test

A.3 Session Management Testing

Session cookie inspection

The screenshot displays the Burp Suite interface with the HTTP history tab selected. The history table shows a POST request to /login that resulted in a 302 redirect to /dashboard. The response headers in the Inspector panel show a Set-Cookie header: `session=eyJlc2...` with attributes `HttpOnly` and `Path=`.

#	Host	Method	URL	Params	Edited	Status	Content Length	MIME type	Extensi Title	Notes	TLS IP	Cookies
402	http://192.168.1.5000	GET	/favicon.ico			404	402	HTML	ico	404 Not Found	192.168.1.5000	
401	http://192.168.1.5000	GET	/static/js/app.js			304	309	script	js		192.168.1.5000	
399	http://192.168.1.5000	GET	/dashboard			200	1801	HTML		Dashboard — Northstar Support	192.168.1.5000	
398	http://192.168.1.5000	POST	/login		✓	302	513	HTML		Redirecting...	192.168.1.5000	session=eyJlc2...

Request

```
1 POST /login HTTP/1.1
2 Host: 192.168.1.5000
3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0
4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/svg+xml,*/*;q=0.8
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Content-Type: application/x-www-form-urlencoded
8 Content-Length: 61
9 Origin: http://192.168.1.5000
10 Connection: keep-alive
11 Referer: http://192.168.1.5000/login
12 Upgrade-Insecure-Requests: 1
13 Priority: u=0, i
14 email=dana.brooks%40example.invalid&password=LabAdmin-2026%21
```

Response

```
1 HTTP/1.1 302 FOUND
2 Server: Werkzeug/3.1.8 Python/3.12.3
3 Date: Sun, 30 Aug 2026 20:47:01 GMT
4 Content-Type: text/html; charset=utf-8
5 Content-Length: 207
6 Location: /dashboard
7 Vary: Cookie
8 Set-Cookie: session=eyJlc2...
9 Connection: close
10
11 <!doctype html>
12 <html lang=en>
13 <title>Redirecting...</title>
14 <h1>Redirecting...</h1>
15 <p>You should be redirected automatically to the target URL: <a href="/dashboard">/dashboard</a>. If not, click the link.
16
```

Inspector

Response header

Name	Value
Set-Cookie	session=eyJlc2...; HttpOnly; Path=

Figure 9. Set-Cookie header showing session cookie name and attributes

Session binding and tamper resistance

The screenshot displays the Burp Suite Community Edition v2024.9.4 interface. The top menu bar includes Burp, Project, Intruder, Repeater, View, and Help. Below it, a toolbar shows various tools like Dashboard, Target, Proxy, Intruder, Repeater, Collaborator, Sequencer, Decoder, Comparer, Logger, Organizer, Extensions, and Learn. The main workspace is divided into two panels: Request and Response.

Request Panel:

```
1 GET /login HTTP/1.1
2 Host: 192.168.1.1:5000
3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0
4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/png,image/svg+xml,*/*;q=0.8
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Connection: keep-alive
8 Upgrade-Insecure-Requests: 1
9 Priority: u=0, i
10 Cookie: session=FIXED123456789SS
11
12
```

Response Panel:

```
10 <html lang="en">
11 <head>
12   <meta charset="utf-8">
13   <meta name="viewport" content="width=device-width, initial-scale=1">
14   <title>Sign in - Northstar Support</title>
15   <link rel="stylesheet" href="/static/css/styles.css">
16 </head>
17 <body>
18   <header class="site-header">
19     <div class="container nav">
20       <a class="brand" href="/">Northstar Support</a>
21       <nav>
22         <a href="/">Home</a>
23         <a href="/login">Sign in</a>
24       </nav>
25     </div>
26   </header>
27   <main class="container">
28     <section class="form-card">
29       <p class="eyebrow">Employee portal</p>
30       <h1>Sign in</h1>
31       <form method="post" action="/login">
32         <label for="email">Email address</label>
33         <input
34           id="email"
35           name="email"
36           type="email"
37           autocomplete="username"
38           required
39         >
40         <label for="password">Password</label>
41         <input
42           id="password"
43           name="password"
44           type="password"
45           autocomplete="current-password"
46           required
47         >
48       </form>
49     </section>
50   </main>
51 </body>
52 </html>
```

Figure 10a. Request with a custom session cookie and the application's response

Burp Suite Community Edition v2024.9.4 - Temporary Project

Dashboard Target Proxy Intruder Repeater Collaborator Sequencer Decoder Comparer Logger Organizer Extensions Learn

Intercept HTTP history WebSockets history Match and replace Proxy settings

Filter settings: Hiding CSS, image and general binary content

#	Host	Method	URL	Params	Edited	Status	code	Length	MIME type	Extensi Title	Notes	TLS IP	Cookies	Time
483	http://192.168.1.100	GET	/favicon.ico			404	402	HTML	ico	404 Not Found		192.168.1.100		22:18:08 31 Aug 2026
482	http://192.168.1.100	GET	/static/js/app.js			304	309	script	js			192.168.1.100		22:18:08 31 Aug 2026
480	http://192.168.1.100	GET	/dashboard			200	1802	HTML		Dashboard — Northstar Support		192.168.1.100		22:18:07 31 Aug 2026
479	http://192.168.1.100	POST	/login		✓	302	513	HTML		Redirecting...		192.168.1.100	session=eyJlc2...	22:18:05 31 Aug 2026
478	http://192.168.1.100	GET	/static/js/app.js			304	309	script	js			192.168.1.100		22:17:47 31 Aug 2026
476	http://192.168.1.100	GET	/login			200	1733	HTML		Sign in — Northstar Support		192.168.1.100		22:17:45 31 Aug 2026
475	http://192.168.1.100	GET	/favicon.ico			404	402	HTML	ico	404 Not Found		192.168.1.100		22:17:42 31 Aug 2026
474	http://192.168.1.100	GET	/static/js/app.js			200	474	script	js			192.168.1.100		22:17:42 31 Aug 2026
472	http://192.168.1.100	GET	/			200	1824	HTML		Northstar Support		192.168.1.100		22:17:39 31 Aug 2026

Request

```

1 POST /login HTTP/1.1
2 Host: 192.168.1.100:5000
3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0
4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/png,image/svg+xml,*/*;q=0.8
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Content-Type: application/x-www-form-urlencoded
8 Content-Length: 62
9 Origin: http://192.168.1.100:5000
10 Connection: keep-alive
11 Referer: http://192.168.1.100:5000/login
12 Upgrade-Insecure-Requests: 1
13 Priority: u=0, i
14
15 email=alice.morgan%40example.invalid&password=LabAlice-2026%21

```

Response

```

1 HTTP/1.1 302 FOUND
2 Server: Werkzeug/3.1.8 Python/3.12.3
3 Date: Tue, 01 Sep 2026 01:18:05 GMT
4 Content-Type: text/html; charset=utf-8
5 Content-Length: 207
6 Location: /dashboard
7 Vary: Cookie
8 Set-Cookie: session=eyJlc2...; HttpOnly; Path=/
9 Connection: close
10
11 <!doctype html>
12 <html lang=en>
13 <title>Redirecting...</title>
14 <h1>Redirecting...</h1>
15 <p>You should be redirected automatically to the target URL: <a href="/dashboard">/dashboard</a>. If not, click the link.
16

```

Inspector

Response header

Name

Set-Cookie

Value

session=eyJlc2...; HttpOnly; Path=

Figure 10b. Rejection of an invalid/tampered session cookie

Logout and session invalidation

Burp Suite Community Edition v2024.9.4 - Temporary Project

Dashboard Target Proxy Intruder Repeater Collaborator Sequencer Decoder Comparer Logger Organizer Extensions Learn

1 x 2 x 3 x 4 x 5 x 6 x +

Send Cancel Follow redirection

Request

```

1 POST /login HTTP/1.1
2 Host: 192.168.1.100:5000
3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0
4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/png,image/svg+xml,*/*;q=0.8
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Content-Type: application/x-www-form-urlencoded
8 Content-Length: 62
9 Origin: http://192.168.1.100:5000
10 Connection: keep-alive
11 Referer: http://192.168.1.100:5000/login
12 Cookie: session=eyJlc2...; n1eL4
13 Upgrade-Insecure-Requests: 1
14 Priority: u=0, i
15
16 email=alice.morgan%40example.invalid&password=LabAlice-2026%21

```

Response

```

1 HTTP/1.1 302 FOUND
2 Server: Werkzeug/3.1.8 Python/3.12.3
3 Date: Fri, 04 Sep 2026 17:42:06 GMT
4 Content-Type: text/html; charset=utf-8
5 Content-Length: 207
6 Location: /dashboard
7 Vary: Cookie
8 Set-Cookie: session=eyJlc2...; 980nMg; HttpOnly; Path=/
9 Connection: close
10
11 <!doctype html>
12 <html lang=en>
13 <title>Redirecting...</title>
14 <h1>Redirecting...</h1>
15 <p>You should be redirected automatically to the target URL: <a href="/dashboard">/dashboard</a>. If not, click the link.
16

```

Figure 10c. Post-logout behavior and verification that the previous session token is no longer accepted

A.4 Authorization and Access Control Testing

Horizontal privilege escalation (IDOR) on tickets

The screenshot displays the Burp Suite interface. At the top, the title bar reads "Burp Suite Community Edition v2024.9.4 - Temporary Project". The menu bar includes "Burp", "Project", "Intruder", "Repeater", "View", and "Help". The toolbar contains buttons for "Dashboard", "Target", "Proxy" (selected), "Intruder", "Repeater", "Collaborator", "Sequencer", "Decoder", "Comparer", "Logger", "Organizer", "Extensions", and "Learn". Below the toolbar, a status bar indicates "Logging of out-of-scope Proxy traffic is disabled" with a "Re-enable" button. A filter settings bar shows "Filter settings: Hiding CSS, image and general binary content".

The HTTP history table shows the following entries:

#	Host	Method	URL	Params	Edited	Status code	Length	MIME type	Extension	Title	Notes	T
369	http://192.168.1.100:5000	GET	/static/js/app.js			304	309	script	js			
367	http://192.168.1.100:5000	GET	/tickets/1			200	1469	HTML		Ticket 1 — Northstar Support		

The selected request (367) is shown in the "Request" pane. The "Pretty" tab is active, displaying the following details:

```
1 GET /tickets/1 HTTP/1.1
2 Host: 192.168.1.100:5000
3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0
4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/png,image/svg+xml,*/*;q=0.8
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Connection: keep-alive
8 Cookie: session=eyJlc2... MhsgA
9 Upgrade-Insecure-Requests: 1
10 Priority: u=0, i
11
12
```

The selected response (367) is shown in the "Response" pane. The "Pretty" tab is active, displaying the following HTML content:

```
33 Ticket #1: Alice ticket 1
34 </h1>
35 <p>
36 This is Alice's first ticket.
37 </p>
38 <p>
39 <strong>
40 Priority:
41 medium |
42 </strong>
43 <strong>
44 Status:
45 open |
46 </strong>
47 <strong>
48 Owner:
49 1 |
50 </strong>
51 <strong>
52 Assigned:
53 1
54 </strong>
55 </p>
56 <p>
57 <strong>
58 Created:
59 2026-09-04 20:18:04 |
60 </strong>
61 <strong>
62 Updated:
63 2026-09-04 20:18:04
64 </strong>
65 </p>
66 </section>
67
68 </main>
```

Figure 11a. Alice logged in and viewing her ticket (/tickets/1)

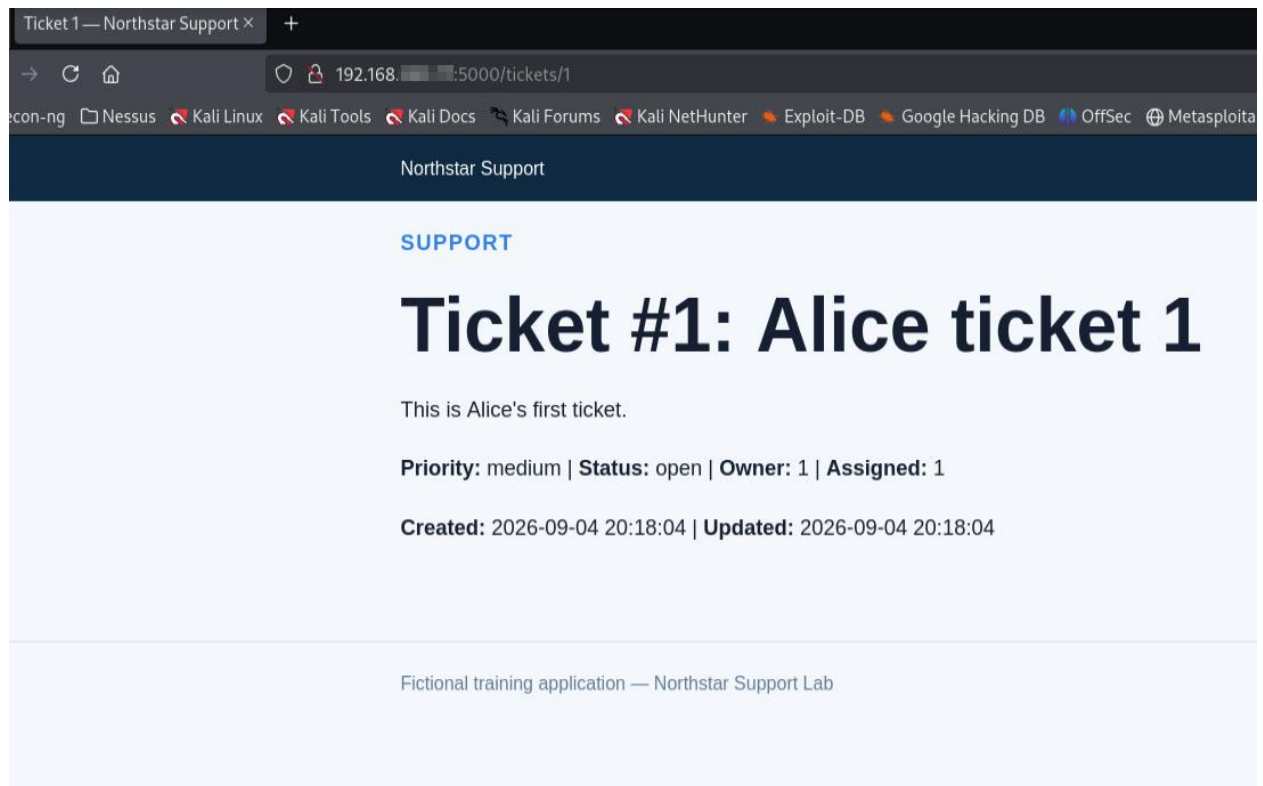


Figure 11b. Alice's ticket list (baseline view)

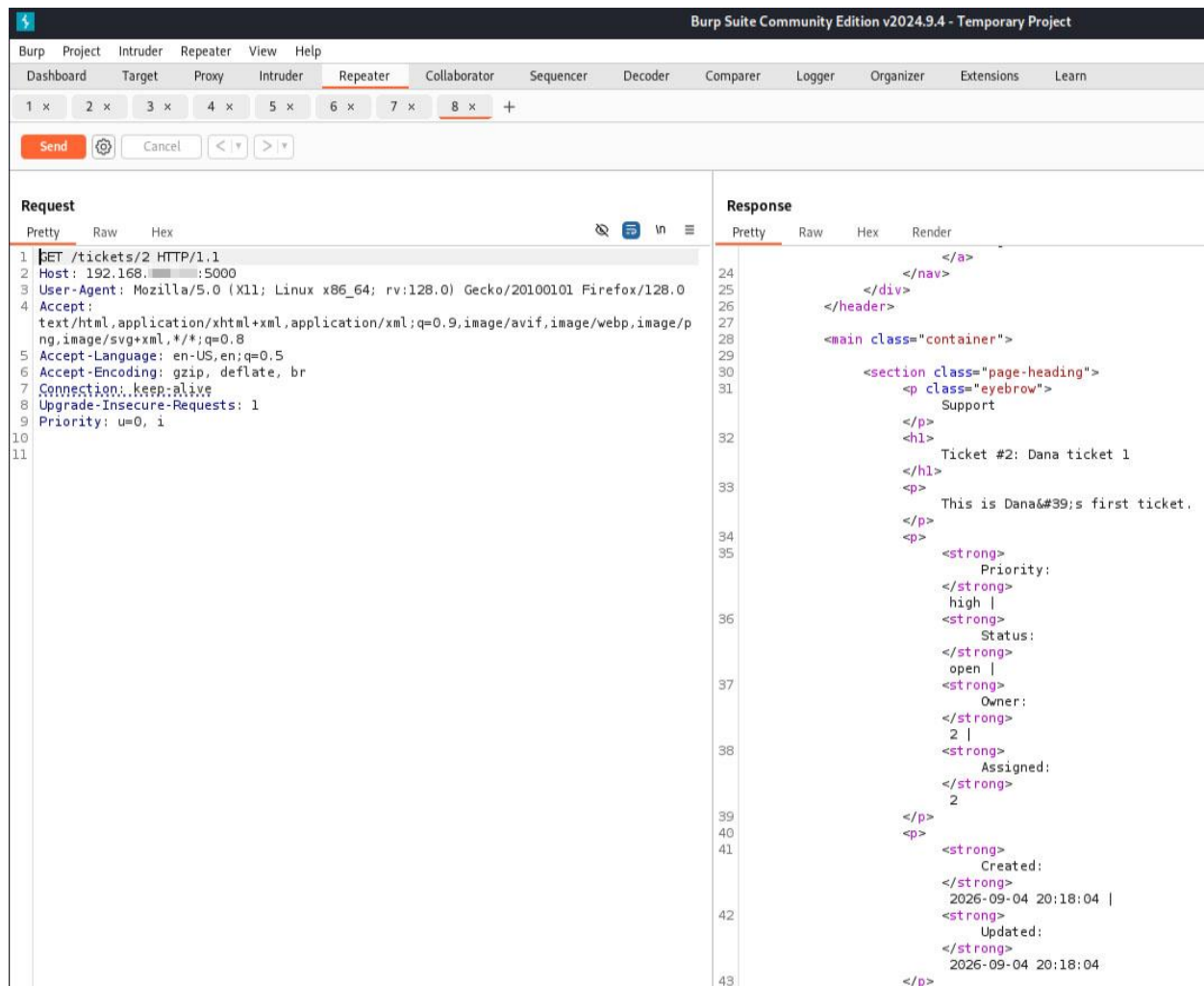


Figure 12. Access to another user's ticket by manipulating the ticket identifier (/tickets/2)

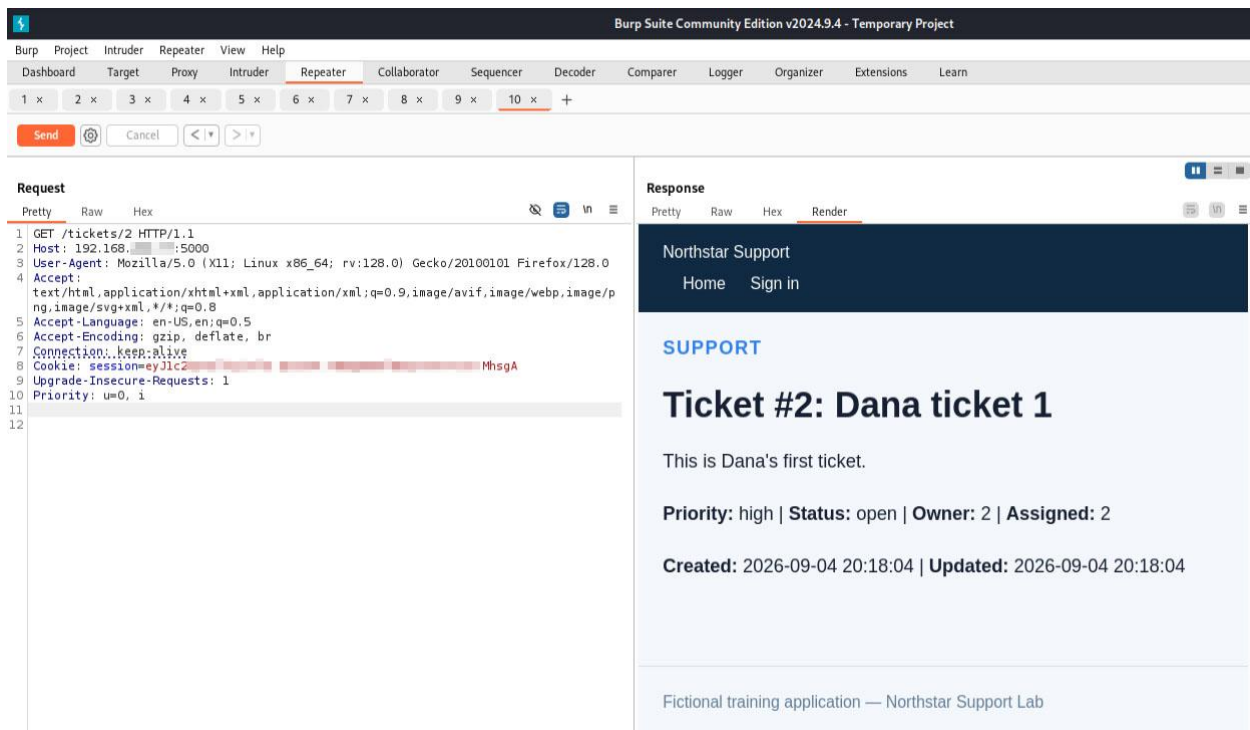


Figure 13. Repeater view confirming IDOR with rendered response

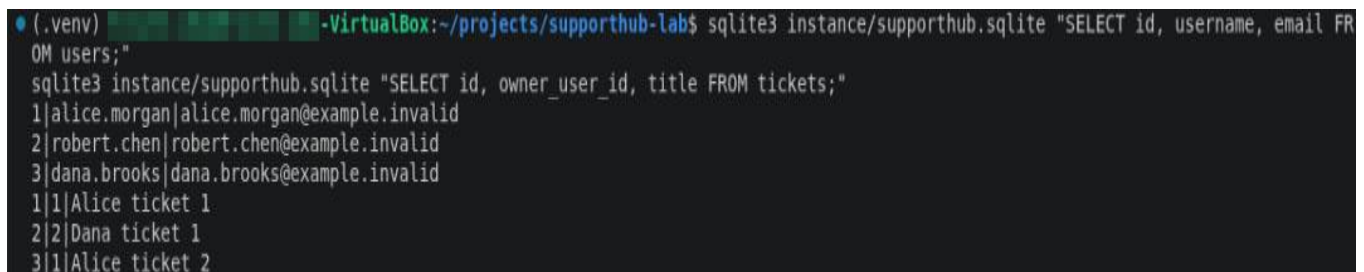


Figure 14. SQLite view of the users table, confirming multiple accounts associated with tickets

A.5 Additional Tests (Negative Results)

SQL injection and XSS screening

Login-related Repeater screenshots (see Section A.2) were used to verify application responses to basic SQLi/XSS probes. No distinct additional screenshots were required for these negative results.

Vertical privilege escalation (admin route)

The admin-route test returned 403 Forbidden, indicating enforced access control. This behavior is described in the main report; no separate screenshot is included.

Appendix B: Test Limitations

The following limitations define the depth and interpretation of this assessment.

B.1 Scope and Coverage

- The assessment was time-boxed and focused on high-impact web-application controls, principally authentication, session management, and authorization.
- Not every endpoint, HTTP method, parameter, user role, workflow, or error condition was exhaustively tested.
- The assessment was conducted against the specified Northstar lab deployment; findings may not apply unchanged to another environment or production deployment.

B.2 Input Validation

- SQL injection testing consisted of basic screening on the login flow and selected inputs. Blind, time-based, second-order, and database-specific techniques were not comprehensively tested.
- XSS testing consisted of basic reflected-input screening. Stored XSS, DOM-based XSS, and context-specific encoding bypasses were not comprehensively assessed.
- CSRF defenses were not subject to dedicated, end-to-end validation.

B.3 Authentication and Sessions

- Password-storage details, including hashing algorithm, salt use, work factor, and password-reset flow security, were not independently verified.
- Intruder testing was limited to avoid account lockout, excessive authentication traffic, and disruption. A large-scale credential attack was not performed.
- Cookie behavior was evaluated from observed HTTP responses; infrastructure-level TLS termination, reverse-proxy configuration, and production security controls were not independently reviewed.

B.4 Authorization and Environment

- Access-control testing confirmed the documented ticket IDOR scenario, but it did not constitute exhaustive authorization testing across every resource type.
- Administrative functionality was tested only at the endpoint(s) available within the approved scope. The observed 403 Forbidden result is evidence for that tested route only.
- Internal network segmentation, host hardening, source-code review, dependency analysis, cloud configuration review, and post-compromise activities were outside this web assessment's scope.

B.5 Interpretation

Negative results mean that no vulnerability was identified through the techniques and coverage described above. They do not demonstrate that the application is free of vulnerabilities in those categories.

Appendix C: References

1. **OWASP Web Security Testing Guide.** Methodology and test cases for web-application security testing.
<https://owasp.org/www-project-web-security-testing-guide/>
2. **OWASP Top 10: 2025.** Includes Broken Access Control and Identification and Authentication Failures.
<https://top10.owasp.org/2025/>
3. **OWASP Authorization Cheat Sheet.** Guidance for server-side authorization and deny-by-default design.
<https://cheatsheetseries.owasp.org/cheatsheets/Authorization Cheat Sheet.html>
4. **OWASP Insecure Direct Object Reference Prevention Cheat Sheet.** Guidance for preventing object-level authorization failures.
<https://cheatsheetseries.owasp.org/cheatsheets/Insecure Direct Object Reference Prevention Cheat Sheet.html>
5. **OWASP Session Management Cheat Sheet.** Guidance on session identifiers, cookies, transport protection, and logout handling.
<https://cheatsheetseries.owasp.org/cheatsheets/Session Management Cheat Sheet.html>
6. **OWASP Authentication Cheat Sheet.** Guidance on authentication design, credential handling, and defenses against automated attacks.
<https://cheatsheetseries.owasp.org/cheatsheets/Authentication Cheat Sheet.html>
7. **PortSwigger Web Security Academy: Access Control Vulnerabilities.** Background on vertical and horizontal privilege escalation and IDOR testing.
<https://portswigger.net/web-security/access-control>
8. **PortSwigger Burp Suite Documentation.** Reference for Proxy, Target, Repeater, and Intruder capabilities used during the assessment.
<https://portswigger.net/burp/documentation>